Cyprus University of Technology

CSE 227 Προχωρημένη Αντικειμενοστρεφής Μεθοδολογία και Προγραμματισμός (UML, Java)

Σύνθεση Αντικειμένων, Κληρονομικότητα

Παναγιώτης Ηλία

Μεγάλο Παράδειγμα: Τμήμα πανεπιστημίου

- Θέλουμε να δημιουργήσουμε ένα λογισμικό για ένα τμήμα πανεπιστημίου. Το τμήμα έχει 4 φοιτητές οπού ο καθένας έχει ένα όνομα και ένα αριθμό μητρώου (ΑΜ), και 2 καθηγητές που ο καθένας έχει ένα όνομα και ένα ΑΦΜ. Το τμήμα δίνει 2 μαθήματα. Το κάθε μάθημα έχει κωδικό και όνομα και κάποιες διδακτικές μονάδες. Το κάθε μάθημα ανατίθεται σε ένα καθηγητή. Οι φοιτητές γράφονται σε κάποιο μάθημα και αν περάσουν το μάθημα παίρνουν τις μονάδες. Θέλουμε να μπορούμε να τυπώσουμε τις πληροφορίες για το μάθημα: το όνομα, τον καθηγητή και τη λίστα των φοιτητών που παίρνουν το μάθημα.

Μεγάλο Παράδειγμα: Τμήμα πανεπιστημίου

- Θέλουμε να δημιουργήσουμε ένα λογισμικό για ένα **τμήμα** πανεπιστημίου.
- Το τμήμα έχει 4 **φοιτητές** οπού ο καθένας έχει **όνομα** και **αριθμό μητρώου** (ΑΜ).
- Το τμήμα έχει 2 **καθηγητές** που ο καθένας έχει **όνομα** και **ΑΦΜ**.
- Το τμήμα δίνει 2 **μαθήματα**. Το κάθε μάθημα έχει **κωδικό** και **όνομα**, και κάποιες **διδακτικές μονάδες**.
- Το κάθε μάθημα **ανατίθεται** σε ένα καθηγητή.
- Οι φοιτητές **γράφονται** σε κάποιο μάθημα και αν **περάσουν** θα **πάρουν** τις μονάδες.
- Θέλουμε να μπορούμε να **τυπώσουμε** τις πληροφορίες του μαθήματος: το **όνομα**, τον **καθηγητή** και τη **λίστα** των **φοιτητών** που παίρνουν το μάθημα.

Κλάσεις, μέθοδοι και πεδία

- Ουσιαστικά:
  - **Τμήμα**
  - **Φοιτητές**
  - **Καθηγητές**
  - **Μαθήματα**
  - **Όνομα**
  - **ΑΜ, ΑΦΜ,**
  - **Κωδικός Μαθήματος**
  - **Βαθμός**
  - **Λίστα φοιτητών**
- Τα ουσιαστικά είναι υποψήφια για κλάσεις ή πεδία

- Ρήματα:
  - **Ανατίθεται**
  - **Εγγράφεται**
  - **Τυπώνει**
  - **Περνάω μάθημα**
  - **Παίρνω μονάδες**
- Τα ρήματα είναι υποψήφια για να γίνουν μέθοδοι και μηνύματα μεταξύ αντικειμένων.

Κλάσεις μέθοδοι και πεδία

- Ουσιαστικά:
  - **Τμήμα**
  - **Φοιτητές**
  - **Καθηγητές**
  - **Μαθήματα**
  - **Όνομα**
  - **ΑΜ, ΑΦΜ,**
  - **Κωδικός Μαθήματος**
  - **Βαθμός**
  - **Λίστα φοιτητών**
- Τα ουσιαστικά είναι υποψήφια για κλάσεις ή πεδία

- Ρήματα:
  - **Ανατίθεται**
  - **Εγγράφεται**
  - **Τυπώνει**
  - **Περνάω μάθημα**
  - **Παίρνω μονάδες**
- Τα ρήματα είναι υποψήφια για να γίνουν μέθοδοι και μηνύματα μεταξύ αντικειμένων.

Όλα τα ουσιαστικά μπορούν να γίνουν κλάσεις αλλά συνήθως διαλέγουμε αυτά για τα οποία υπάρχει αρκετή πολυπλοκότητα

Κλάση Professor

- Κρατάει το όνομα και το ΑΦΜ του καθηγητή
- Ενδεχομένως να κρατάει και τα μαθήματα που έχει αναλάβει

- Η μέθοδος για να αναλάβει ο καθηγητής ένα μάθημα θα πρέπει να είναι εδώ ή στην κλάση του μαθήματος?

Κλάση Student

- Κρατάει το όνομα του φοιτητή και τις μονάδες που έχει πάρει μέχρι τώρα
- Ενδεχομένως να κρατάει και τα μαθήματα που παίρνει
- Ενδεχομένως να κρατάει και τη λίστα με τα μαθήματα που έχει περάσει.
- Χρειαζόμαστε μέθοδο για να γραφτεί ο φοιτητής στο μάθημα, ή να το περάσει, ή καλύτερα να τις βάλουμε στην κλάση του μαθήματος?

Κλάση Course

- Κρατάει το όνομα του μαθήματος, τις μονάδες του μαθήματος, τον καθηγητή που κάνει το μάθημα, τους φοιτητές που παίρνουν το μάθημα
  - Τίποτα άλλο? Τι θα κάνουμε με τους βαθμούς και το ποιος πέρασε το μάθημα?
- Μέθοδοι
  - Ανάθεση καθηγητή
  - Εγγραφή φοιτητή στο μάθημα
  - Ανάθεση βαθμών στους φοιτητές

Κλάση Department

- Τα βάζει όλα μαζί, εδώ δημιουργούμε τους φοιτητές, καθηγητές, μαθήματα
- Οι φοιτητές και οι καθηγητές ως άτομα θα μπορούσαν να υπάρχουν και εκτός του τμήματος
- Εδώ δημιουργούμε την main

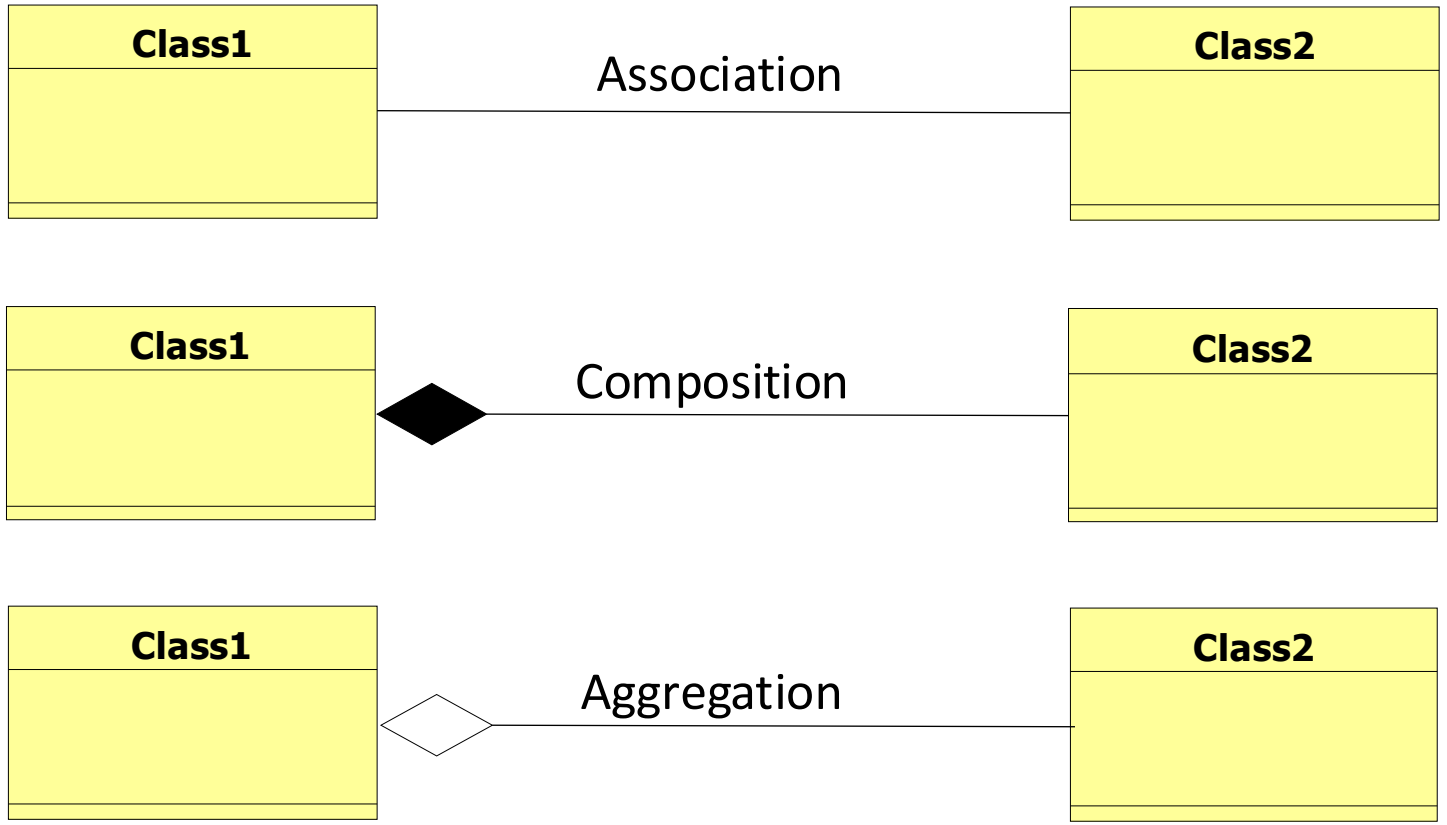
- Χρειαζόμαστε άλλη κλάση?

Κλάση StudentRecord

- Χρειαζόμαστε να κρατάμε για κάθε φοιτητή τις πληροφορίες του (αυτά που έχουμε στο Student class) και το βαθμό του
- Μας βολεύει να δημιουργήσουμε μια καινούρια κλάση που να βάζει μαζί αυτές τις πληροφορίες

UML διαγράμματα

- Association vs Aggregation vs Composition



UML διαγράμματα

- Association

Αν δύο κλάσεις χρειάζεται να “επικοινωνήσουν” μεταξύ τους, πρέπει να υπάρχει σχέση συσχέτισης (association)



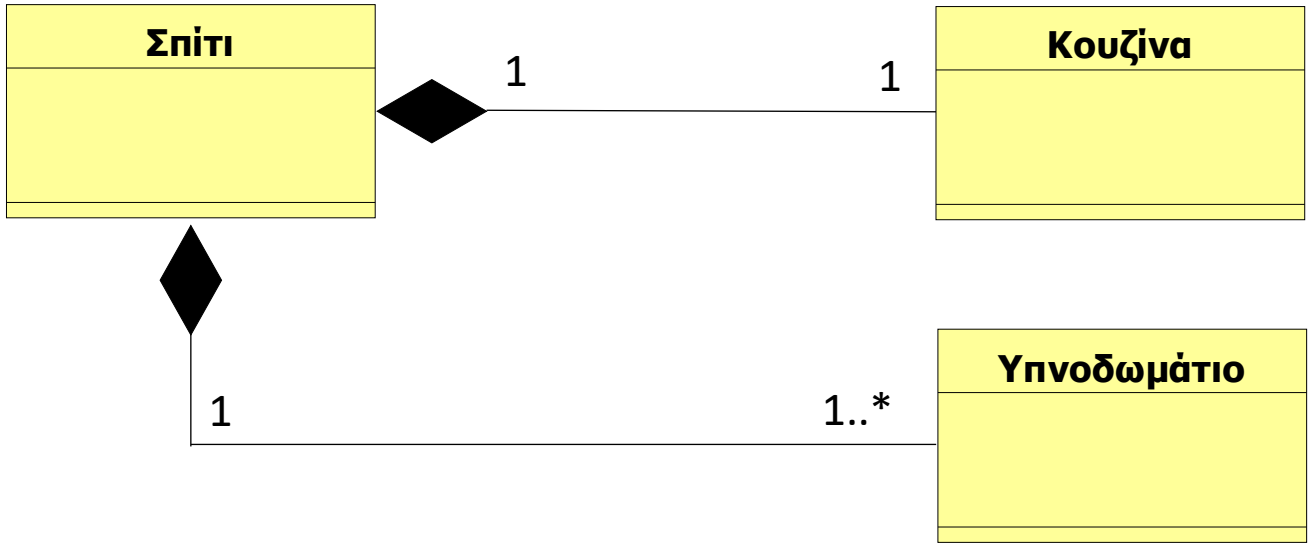
Ένας μαθητής μπορεί να συσχετιστεί με ένα ή περισσότερους δασκάλους

Ένας δάσκαλος μπορεί να συσχετιστεί με ένα ή περισσότερους μαθητές

UML διαγράμματα

- Composition

Υποδηλώνει σχέση στην οποία το παιδί **δεν μπορεί να υπάρξει χωρίς τον πατέρα**. Παράδειγμα: Σπίτι (parent) και δωμάτιο (child). Δεν μπορεί να υπάρξει δωμάτιο χωρίς το σπίτι.

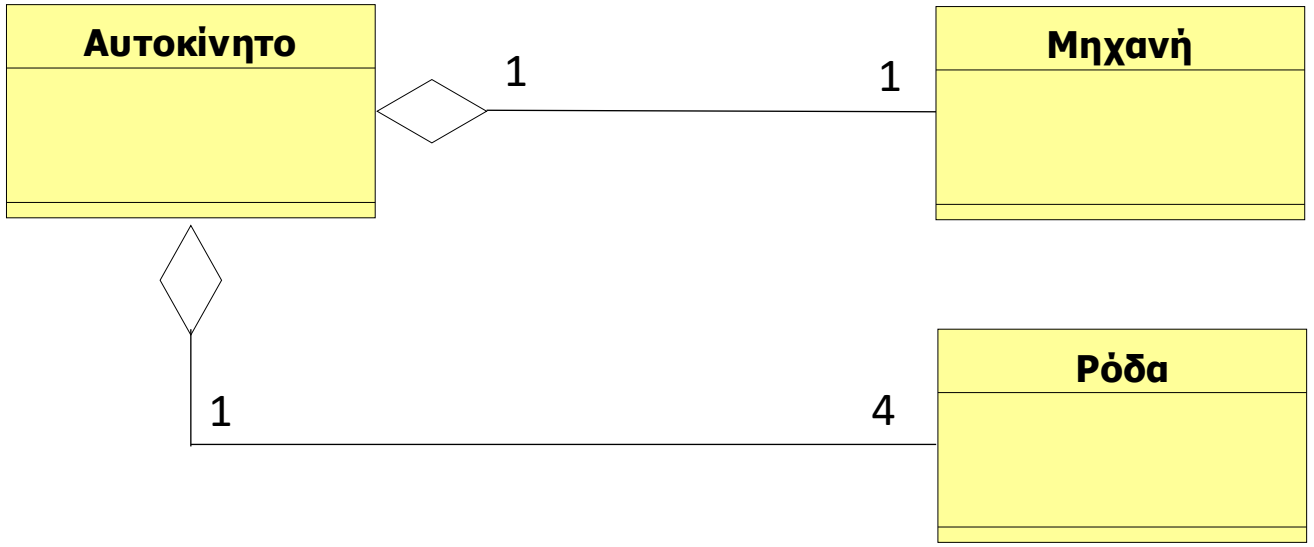


part-of σχέση

UML διαγράμματα

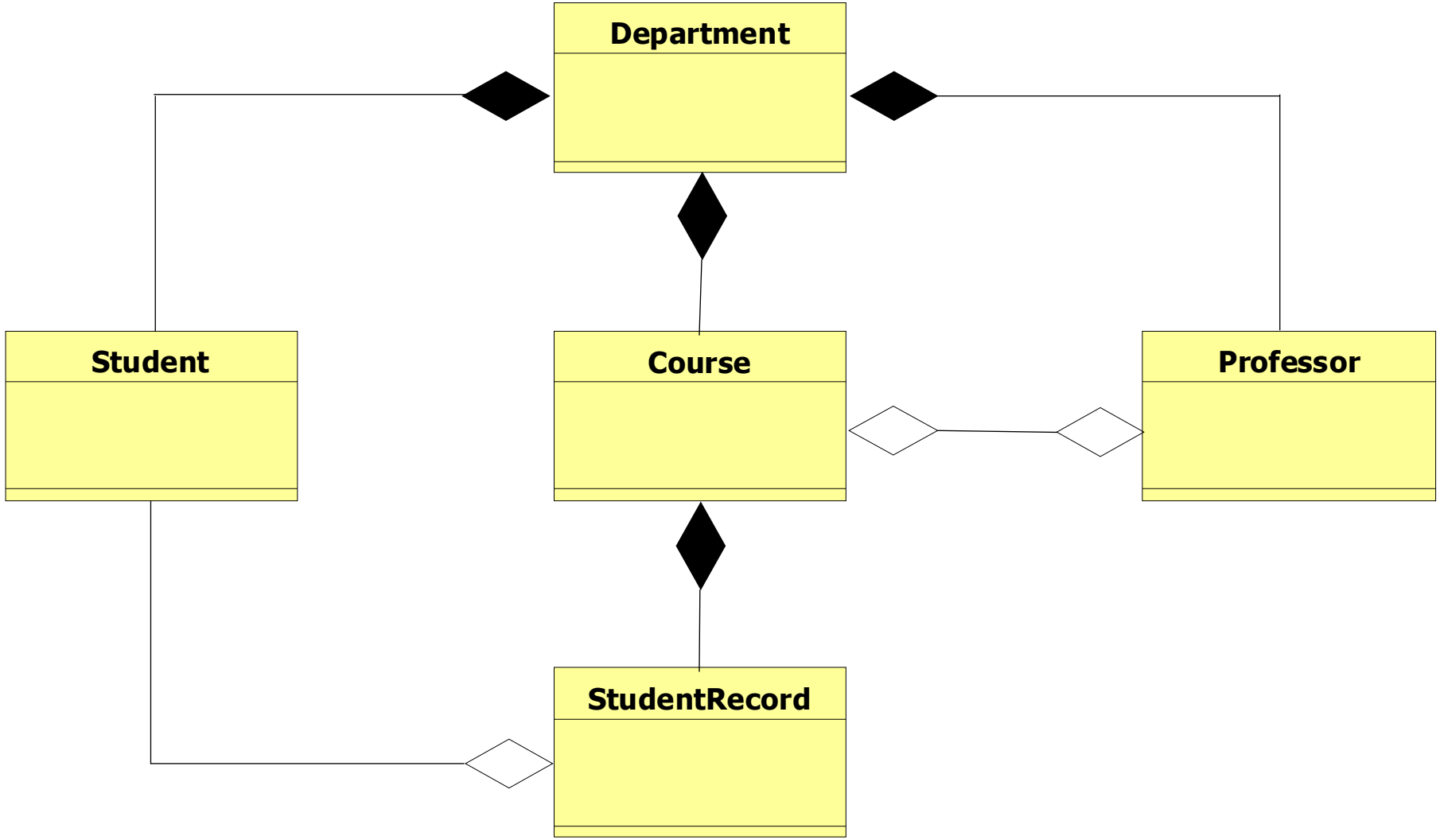
- Aggregation

Υποδηλώνει σχέση συνάθροισης, στην οποία αντικείμενα της μιας κλάσης ”ανήκουν” στο αντικείμενο της άλλης κλάσης, αλλά αυτά **μπορούν να υπάρξουν και όταν το αντικείμενο αυτό καταστραφεί**.



part-of σχέση

Μεγάλο Παράδειγμα - UML διάγραμμα



Αποθήκευση φοιτητών

- Η κλάση Course χρειάζεται να αποθηκεύσει τους φοιτητές που παίρνουν το μάθημα
  - Δεν ξέρουμε εκ των προτέρων πόσοι φοιτητές θα πάρουν το μάθημα
- Θα χρησιμοποιήσουμε την κλάση **ArrayList** για να τους αποθηκεύσουμε

ArrayList

- Το **ArrayList** είναι μια βοηθητική κλάση
  - Είναι ένας δυναμικός πίνακας
    - Προσαρμόζει το μέγεθος του ανάλογα με τον αριθμό των στοιχείων που περιέχει
  - Το **ArrayList** μπορεί να κρατάει αντικείμενα οποιουδήποτε τύπου

- Συντακτικό:
  - `import java.util.ArrayList;`
  - `ArrayList<Τύπος> myList;`

ArrayList

- `ArrayList<Τύπος> myList;`

- Ο τύπος είναι οποιοσδήποτε, μια οποιαδήποτε κλάση.
  - Αυτός είναι ο τύπος των δεδομένων που αποθηκεύει ο πίνακας μας.
  - Για να αποθηκεύσουμε πρωταρχικούς τύπους (int, double, boolean) χρειαζόμαστε την wrapper class.

- Παραδείγματα:
  - `ArrayList<Integer> myList;` // λίστα από ακεραίους
  - `ArrayList<String> myList;` // λίστα από String
  - `ArrayList<Person> myList;` // λίστα από αντικείμενα Person

ArrayList

- Constructor
  - `ArrayList<Type> myList = new ArrayList<Type>();`
- Μέθοδοι
  - `add(Type x)`: προσθέτει το στοιχειό `x` στο τέλος του πίνακα
  - `add(int i, Type x)`: προσθέτει το στοιχειό `x` στη θέση `i` και μετατοπίζει τα υπόλοιπα στοιχεία κατά μια θέση
  - `remove(int i)`: αφαιρεί το στοιχείο στη θέση `i` και το επιστρέφει
  - `remove(Type x)`: αφαιρεί το στοιχείο
  - `set(int i, Type x)`: θέτει στην θέση `i` την τιμή `x` αλλάζοντας την προηγούμενη
  - `get(int i)`: επιστρέφει το αντικείμενο τύπου `Type` στη θέση `i`
  - `size()`: ο αριθμός των στοιχείων του πίνακα
- Διατρέχοντας τον πίνακα
  - `for(Type x: myList){…}`

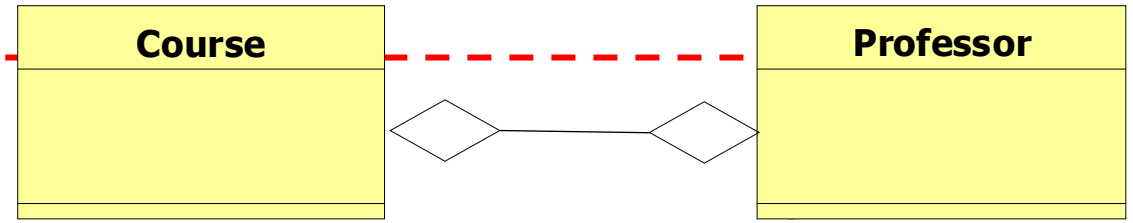
ArrayList

- Διατρέχοντας τον πίνακα:

```
ArrayList<Type> myList = new ArrayList<Type>();
for(Type x: myList){
    System.out.println(x);
}
```

- Εναλλακτικά

```
ArrayList<Type> myList = new ArrayList<Type>();
for(int i=0; i < myList.size(); i++){
    Type x = myList.get(i);
    System.out.println(x);
}
```




```
public class Professor
{
        private String name;
        private int AFM;
        private Course lesson;

        public Professor(String name, int afm){
                this.name = name;
                this.AFM = afm;
        }

        public void setLesson(Course c){
                lesson = c;
        }

        public String toString(){
                return name + " " + AFM + " " + lesson;
        }
}
```

```
public class Student
{
	private String name;
	private int AM;
	private int ECTS = 0;

	public Student(String name, int am){
		this.name = name;
		this.AM = am;
	}

	public int getAM(){
		return AM;
	}

	public void addECTS(int ECTS){
		this.ECTS += ECTS;
	}

	public String toString(){
		return name +" AM:" + AM + " ECTS:" + ECTS;
	}
}
```

```
public class StudentRecord
{
        private Student student;
        private double grade;

        public StudentRecord(Student s){
                student = s;
        }

        public void setGrade(double grade){
                this.grade = grade;
        }

        public Student getStudent(){
                return student;
        }

        public String toString(){
                return student +" :" + grade;
        }

        public boolean passed(){
                if (grade >= 5){ return true;}
                return false;
        }
}
```

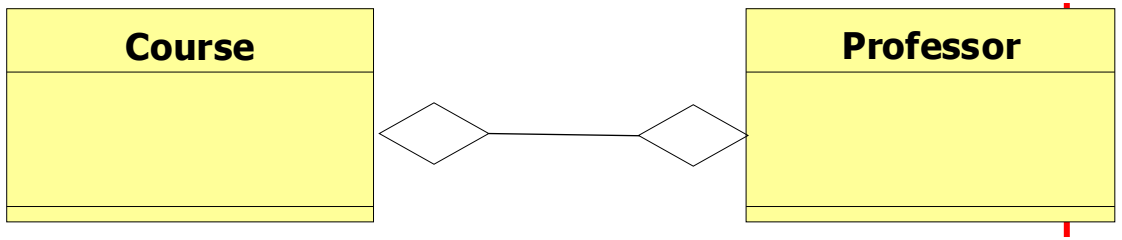
```
import java.util.ArrayList;
import java.util.Scanner;

public class Course
{
        private String name;
        private int code;
        private int ECTS;
        private Professor prof;
        private ArrayList<StudentRecord> studentList = new ArrayList<StudentRecord>();

        public Course(String name, int code, int ECTS){
                this.name = name;
                this.code = code;
                this.ECTS = ECTS;
        }

        public void setProf(Professor p){
                prof = p;
                p.setLesson(this);
        }

        public void enroll(Student s){
                studentList.add(new StudentRecord(s));
        }
```



Χρησιμοποιούμε το this ως αναφορά στο παρόν αντικείμενο, ώστε να το προσθέσουμε στο αντικείμενο Professor

Δημιουργία του StudentRecord και ταυτόχρονη προσθήκη στη λίστα
Λέγεται και «ανώνυμο αντικείμενο»

```
    public void assignGrades(){
        System.out.println("Give grades for course "+toString());
        Scanner input = new Scanner(System.in);
        for(StudentRecord record: studentList){
            System.out.println("Give grade for student " + record.getStudent().getAM() +":");
            double grade = input.nextDouble();
            record.setGrade(grade);
            if (record.passed()){
                record.getStudent().addECTS(ECTS);
            }
        }
    }

    public String toString(){
        return name + " " + code + "("+ECTS + ")";
    }

    public void printInfo(){
        System.out.println("Course " + name +" " + code + "("+ECTS + ")");
        for (StudentRecord r: studentList){
            System.out.println(r);
        }
    }
}
```

Διασχίζουμε τη λίστα των φοιτητών

Αλυσιδωτές κλήσεις μεθόδων
Γίνεται εφόσον μια μέθοδος επιστρέφει αντικείμενο.

```
import java.util.Scanner;
class Department
{
  public static void main(String[] args)
  {
     int numOfStudents = Integer.parseInt(args[0]);

     Professor profX = new Professor("Prof X", 2012);
     Professor profY = new Professor("Prof Y", 2013);

     Course oop = new Course("oop", 227, 6);
     Course intro = new Course("intro", 101, 6);

     Student[] students = new Student[numOfStudents];
     Scanner input = new Scanner(System.in);
     for (int i = 0; i < numOfStudents; i++){
           System.out.print("Give student name: ");
           String name = input.next();
           students[i] = new Student(name, i);
     }

     oop.setProf(profX);
     oop.enroll(students[0]);oop.enroll(students[1]);oop.enroll(students[3]);

     intro.setProf(profY);
     intro.enroll(students[2]); intro.enroll(students[3]);

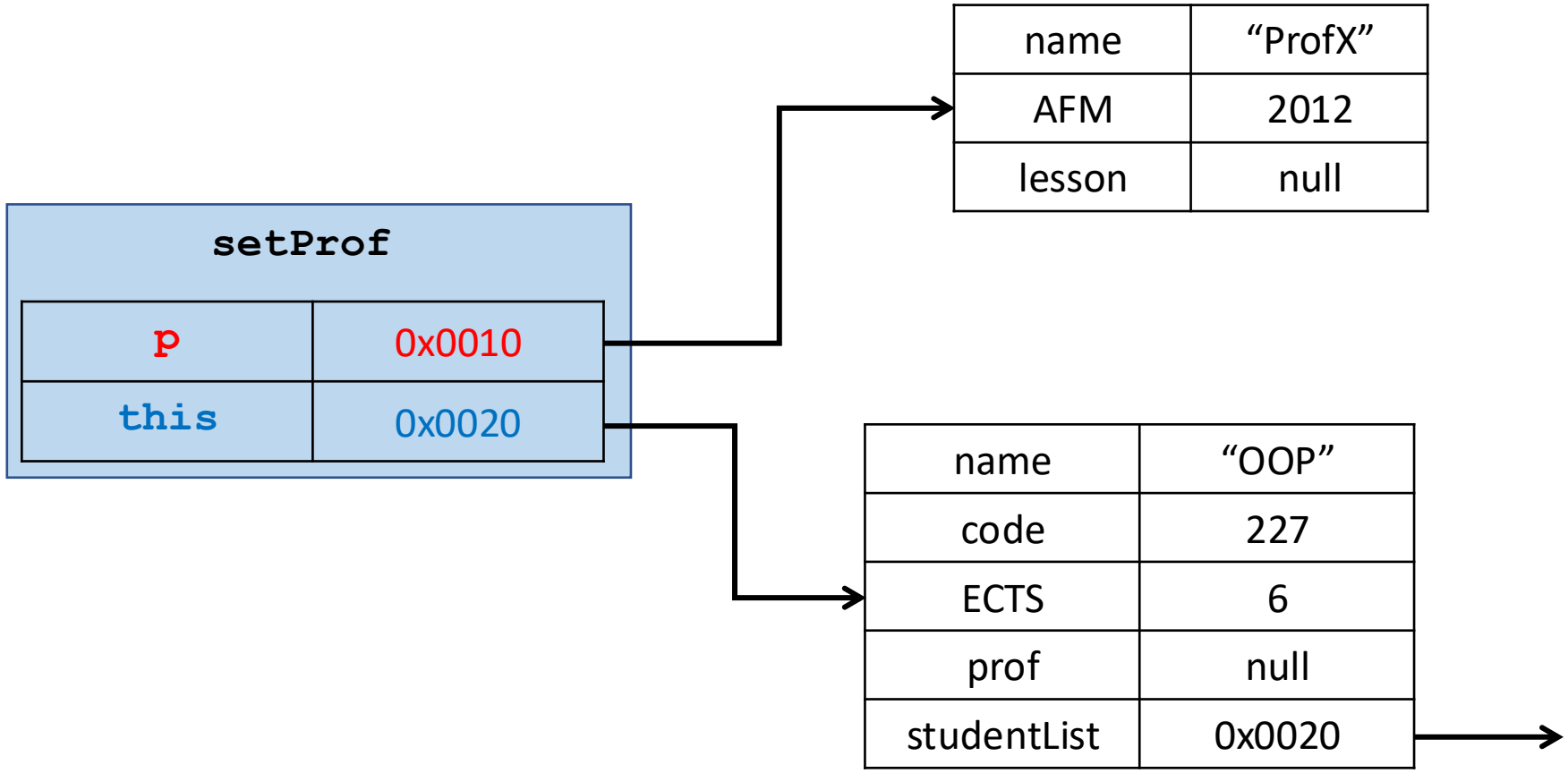
     oop.assignGrades(); intro.assignGrades();

     System.out.println(profX); oop.printInfo();
     System.out.println(profY); intro.printInfo();
  }
}
```

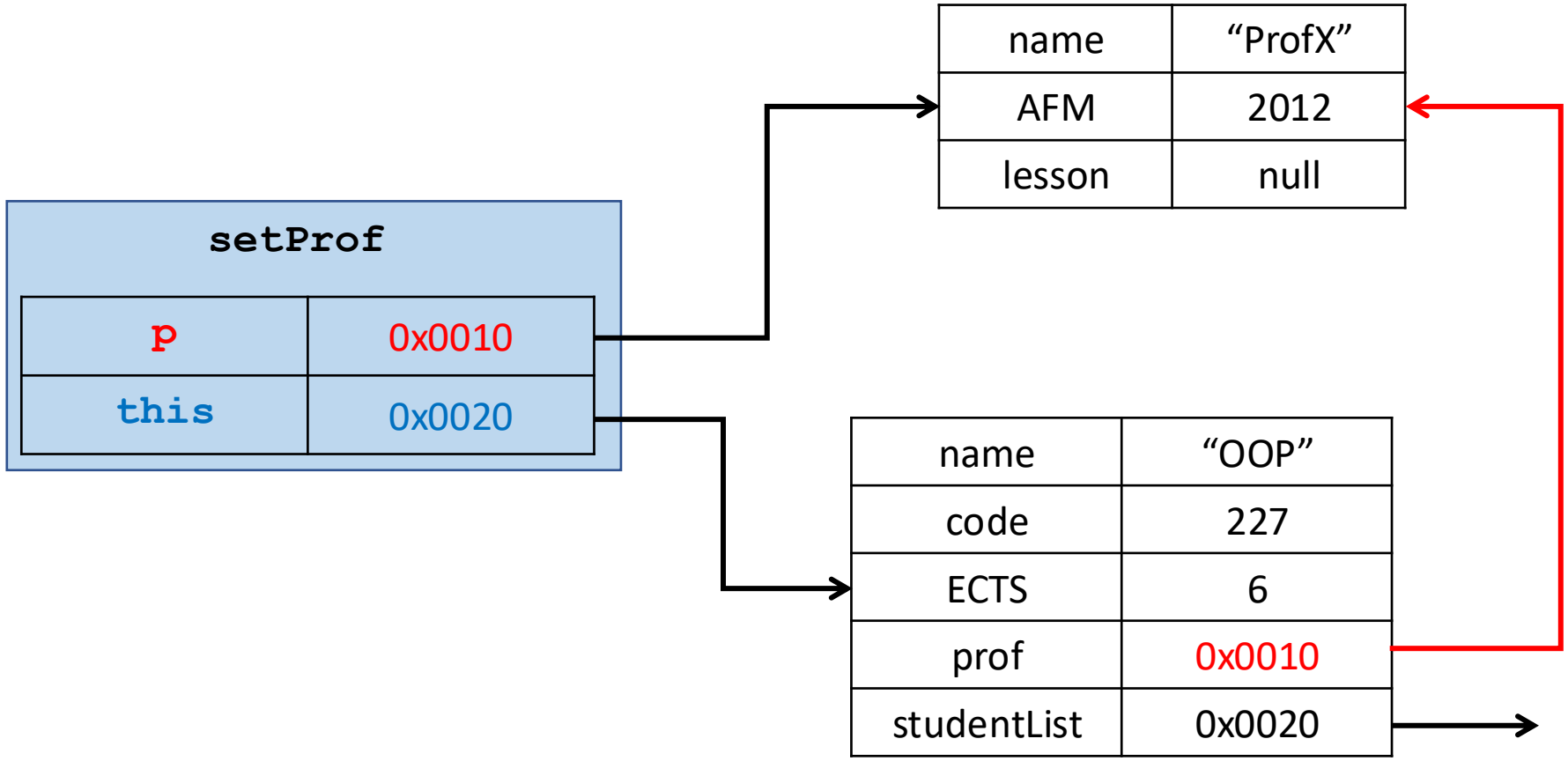
Χρησιμοποιούμε τις παραμέτρους εκτέλεσης (command line arguments) για να περάσουμε τον αριθμό των φοιτητών

Μετατρέπουμε το String σε ακέραιο με την μέθοδο Integer.parseInt

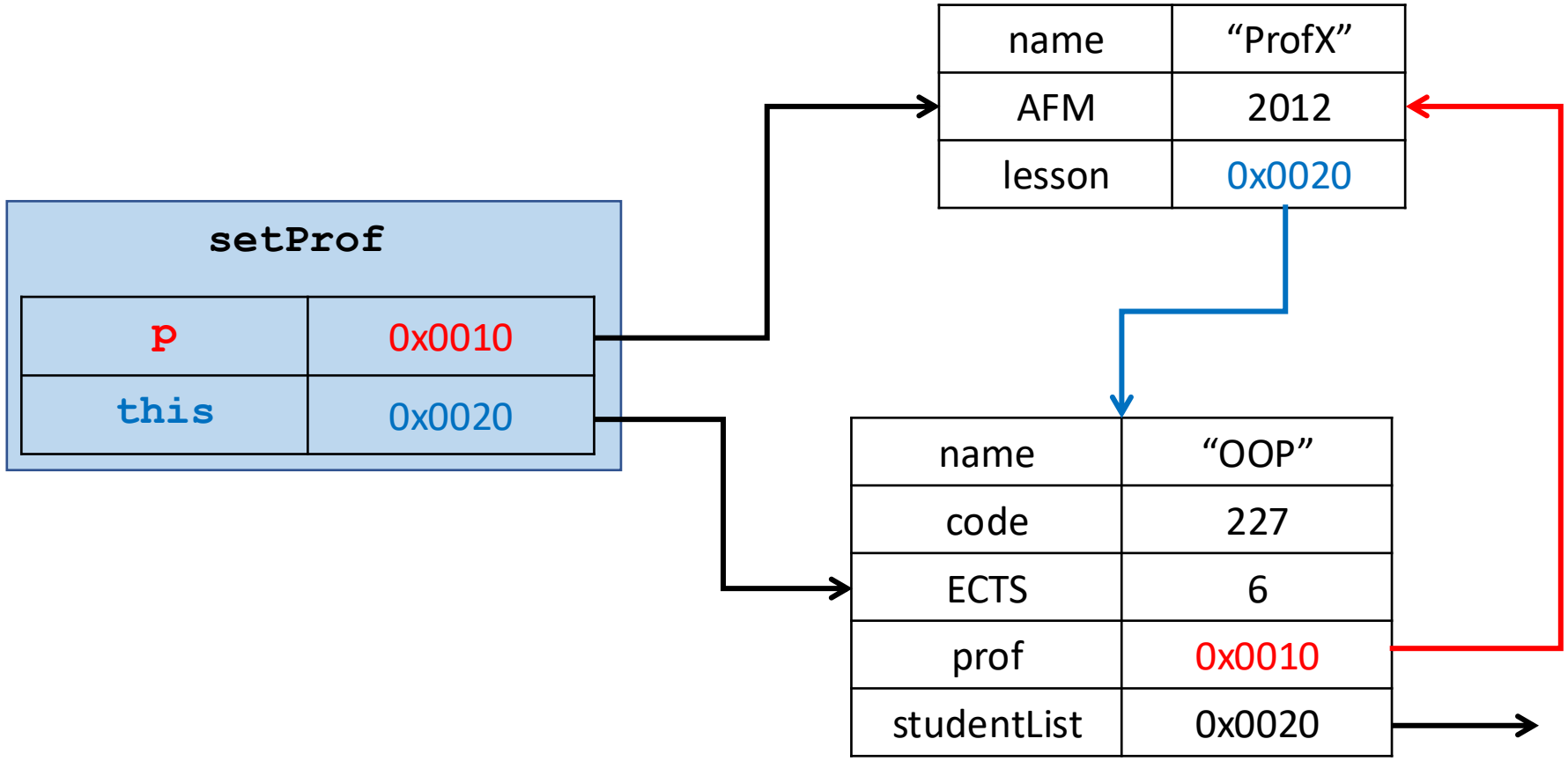
```
public void setProf(Professor p){
        prof = p;
        p.setLesson(this);
}
```



```
public void setProf(Professor p){
        prof = p;
        p.setLesson(this);
}
```



```
public void setProf(Professor p){
        prof = p;
        p.setLesson(this);
}
```



Παράδειγμα της χρήσης της μεταβλητής this

- Έχουμε την κλάση **Person** η οποία κρατάει για κάθε άνθρωπο το όνομα του, το ΑΦΜ του, και την ηλικία του
- Μέσα στην κλάση Person θέλουμε να φτιάξουμε μια μέθοδο `getOlderPerson` η οποία παίρνει σαν όρισμα ένα άλλο Person και επιστρέφει το Person που είναι ηλικιακά μεγαλύτερο

```
public class Person
{
	private String name;
	private int age;

	public Person(String name, int age){
		this.name = name;
		this.age = age;
	}

	public Person getOlderPerson(Person other){
		if(other.age > this.age){
			return other;
		}else{
			return this;
		}
	}
}
```

Η μέθοδος μας επιστρέφει μια αναφορά σε αντικείμενο Person

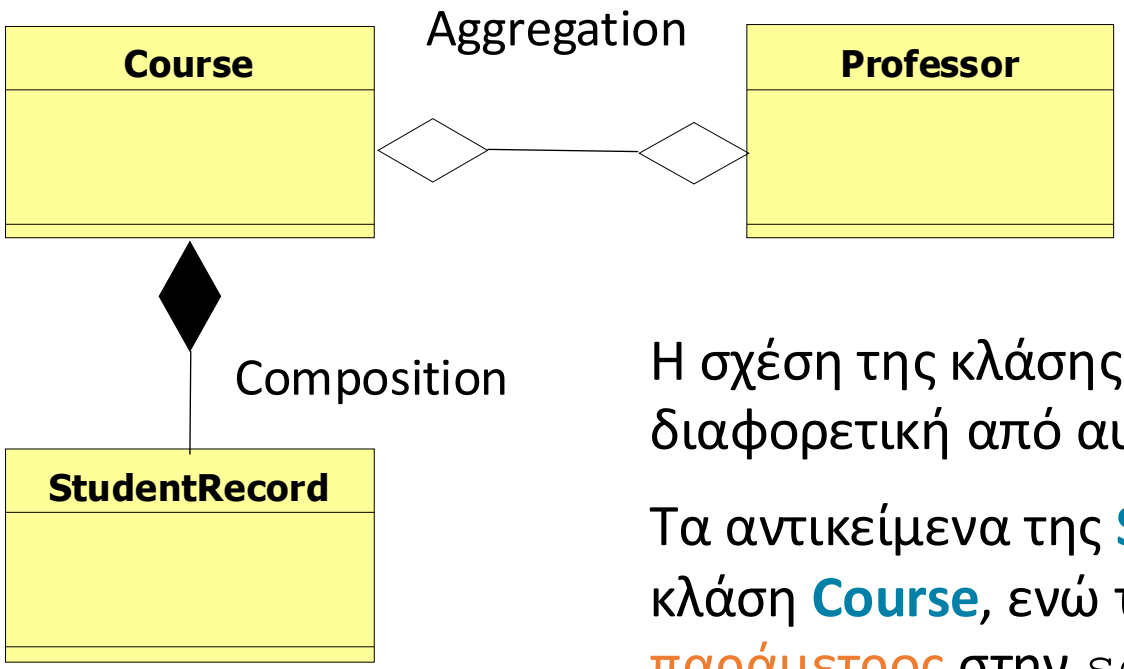
Αν η ηλικία του ατόμου που καλεί την μέθοδο είναι μεγαλύτερη αυτού που περνάμε σαν όρισμα επιστρέφουμε την αναφορά this

Αλλιώς επιστρέφουμε την αναφορά other

Σχέσεις κλάσεων

Προσοχή: Σε πολλά βιβλία και οι δύο σχέσεις αναφέρονται ως σχέση σύνθεσης!

Υπάρχει ποιοτική διαφορά παρότι το όνομα μπορεί να μην διαφέρει



Η σχέση της κλάσης **Course** με την **StudentRecord** είναι διαφορετική από αυτή με την **Professor**

Τα αντικείμενα της **StudentRecord** δημιουργούνται μέσα στην κλάση **Course**, ενώ το αντικείμενο **Professor** περνιέται ως παράμετρος στην `setProf`

Η πρώτη σχέση λέγεται σχέση σύνθεσης (composition) και η δεύτερη σχέση συνάθροισης (aggregation)

Η σχέση **Course** και **Professor** είναι αμφίδρομη μιας και κρατάμε το αντικείμενο **Course** μέσα στην **Professor**

Αν θέλουμε ο φοιτητής να κρατάει πληροφορία για το ποια μαθήματα παίρνει

```
public class Student
{
        private String name;
        private int AM;
        private int ECTS = 0;
        ArrayList<Course> courses = new ArrayList<Course>();

        public Student(String name, int am){
                this.name = name;
                this.AM = am;
        }

        public String getName(){
                return name;
        }

        public void addECTS(int ECTS){
                this.ECTS += ECTS;
        }

        public void addCourse(Course c){
                courses.add(c);
        }

        public String toString(){
                return name +" AM:" + AM + " units:" + units;
        }
}
```

```
import java.util.ArrayList;
import java.util.Scanner;

public class Course
{
        private String name;
        private int code;
        private int ECTS;
        private Professor prof;
        private ArrayList<StudentRecord> studentList = new ArrayList<StudentRecord>();

        public Course(String name, int code, int ECTS){
                this.name = name;
                this.code = code;
                this.ECTS = ECTS;
        }

        public void setProf(Professor p){
                prof = p;
                p.setLesson(this);
        }

        public void enroll(Student s){
                studentList.add(new StudentRecord(s));
                s.addCourse(this);
        }
```

Αναζήτηση

- Τι γίνεται αν θέλουμε να ζητήσουμε τον βαθμό ενός φοιτητή για ένα μάθημα?
  - Η κλάση **Course** θα πρέπει να μπορεί με το ΑΜ του φοιτητή να μας επιστρέφει τον βαθμό
  - Για τέτοιου είδους αναζητήσεις βολεύει να χρησιμοποιούμε ένα λεξικό
  - Η Java μας προσφέρει την κλάση HashMap που υλοποιεί ένα λεξικό

HashMap

- Αποθηκεύει ζευγάρια από **κλειδιά** και **τιμές**
- Constructors
  - `HashMap<TypeK, TypeV> myMap = new HashMap< TypeK, TypeV>();`
- Μέθοδοι
  - `put(TypeK key, TypeV value)`: προσθέτει το ζευγάρι (`key,value`)
  - `TypeV get(TypeK key)`: επιστρέφει την τιμή τύπου `TypeV` για το κλειδί `key`
  - `remove(TypeK key)`: αφαιρεί το ζευγάρι με κλειδί `key`
  - `containsKey(TypeK key)`: boolean αν το σύνολο περιέχει το κλειδί `key` ή όχι
  - `containsValue(TypeV value)`: boolean αν περιέχει την τιμή `value` ή όχι (**αργό**)
  - `size()`: ο αριθμός των στοιχείων (κλειδιών) στο map.
  - `isEmpty()`: boolean αν έχει στοιχεία το map ή όχι
  - `Set<TypeK> keySet()`: επιστρέφει ένα **Set** με τα κλειδιά.
  - `Collection<TypeV> values()`: επιστρέφει ένα **Collection** με τις τιμές

```
import java.util.ArrayList;
import java.util.HashMap;
import java.util.Scanner;

public class Course
{
        private String name;
        private int code;
        private int units;
        private Professor prof;
        private ArrayList<StudentRecord> studentList  = new ArrayList<StudentRecord>();
        private HashMap<Integer, StudentRecord> studentMap = new HashMap<Integer, StudentRecord>();

        public void enroll(Student s){
                studentRecord sRecord = new StudentRecord(s);
                studentList.add(sRecord);
                studentMap.put(s.getAM(),sRecord);
        }

        public double getGrade(int AM){
                if (studentMap.contains(AM)){
                        StudentRecord sRecord = studentMap.get(AM);
                        return sRecord.getGrade();
                }else{
                        System.out.println("Student "+ AM + " not enrolled");
                        return -1;
                }
        }
}
```

Έχοντας το λεξικό μπορούμε να κάνουμε διάφορες αναζητήσεις με το ΑΜ του φοιτητή

Η ίδια αναφορά αποθηκεύεται σε δύο σημεία

Κληρονομικότητα

Παράδειγμα

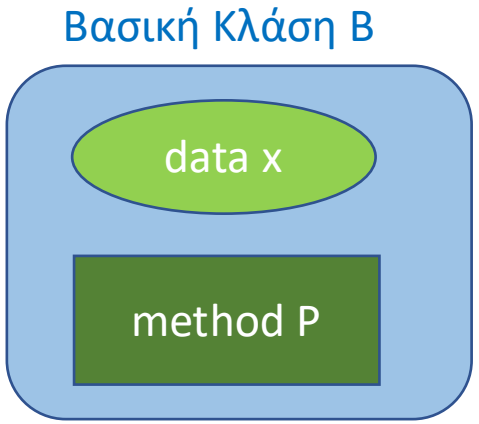
- Στο παράδειγμα με το τμήμα πανεπιστημίου οι φοιτητές και οι καθηγητές είχαν κάποια **κοινά** στοιχεία
  - Και οι δύο είχαν όνομα
  - Και οι δύο είχαν κάποιο χαρακτηριστικό αριθμό και κάποιες **διαφορές**
  - Οι καθηγητές δίδασκαν μαθήματα
  - Οι φοιτητές έπαιρναν μαθήματα, βαθμούς και μονάδες
- Δεν θα ήταν βολικό αν είχαμε μεθόδους που να χειρίζονταν με κοινό τρόπο τις ομοιότητες (π.χ. εκτύπωση των βασικών στοιχείων) και να έχουν ξεχωριστές μεθόδους για τις διαφορές?
  - Έτσι δεν θα έπρεπε να γράφουμε τον ίδιο κώδικα πολλές φορές και οι αλλαγές θα έπρεπε να γίνουν μόνο μια φορά
- Αυτό το καταφέρνουμε με την κληρονομικότητα!

Κληρονομικότητα

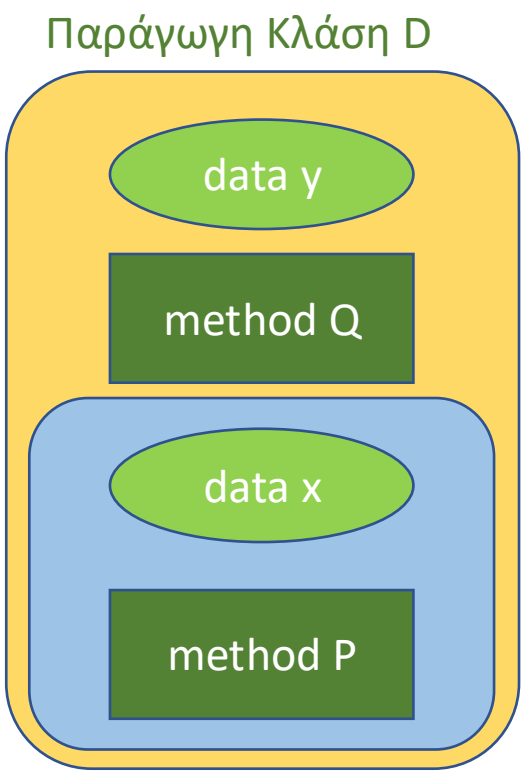
- Η κληρονομικότητα είναι κεντρική έννοια στον αντικειμενοστραφή προγραμματισμό
- Θα ορίσουμε μια **γενική κλάση** που έχει κάποια χαρακτηριστικά (πεδία και μεθόδους) που θέλουμε και μετά να ορίσουμε **εξειδικευμένες παραλλαγές** της κλάσης αυτής στις οποίες προσθέτουμε ειδικότερα χαρακτηριστικά
  - Οι εξειδικευμένες κλάσεις λέμε ότι κληρονομούν τα χαρακτηριστικά της γενικής κλάσης

Κληρονομικότητα

Έχουμε μια **Βασική Κλάση (Base Class) Β**, με κάποια πεδία και μεθόδους



Θέλουμε να δημιουργήσουμε μια **νέα κλάση D** η οποία να έχει όλα τα χαρακτηριστικά της Β, **αλλά και κάποια επιπλέον**



Αντί να ξαναγράψουμε τον ίδιο κώδικα δημιουργούμε μια **Παράγωγη Κλάση (Derived Class) D**, η οποία κληρονομεί όλη τη λειτουργικότητα της **Βασικής Κλάσης Β** και στην οποία προσθέτουμε τα νέα πεδία και μεθόδους

Αυτή διαδικασία λέγεται κληρονομικότητα

Κληρονομικότητα

- Η κληρονομικότητα είναι χρήσιμη όταν
  - Θέλουμε να έχουμε αντικείμενα και της **κλάσης Β** και της **κλάσης D**
  - Θέλουμε να ορίσουμε **πολλαπλές παράγωγες κλάσεις D1, D2, …** που η κάθε μία επεκτείνει την **Β** με **διαφορετικό τρόπο**

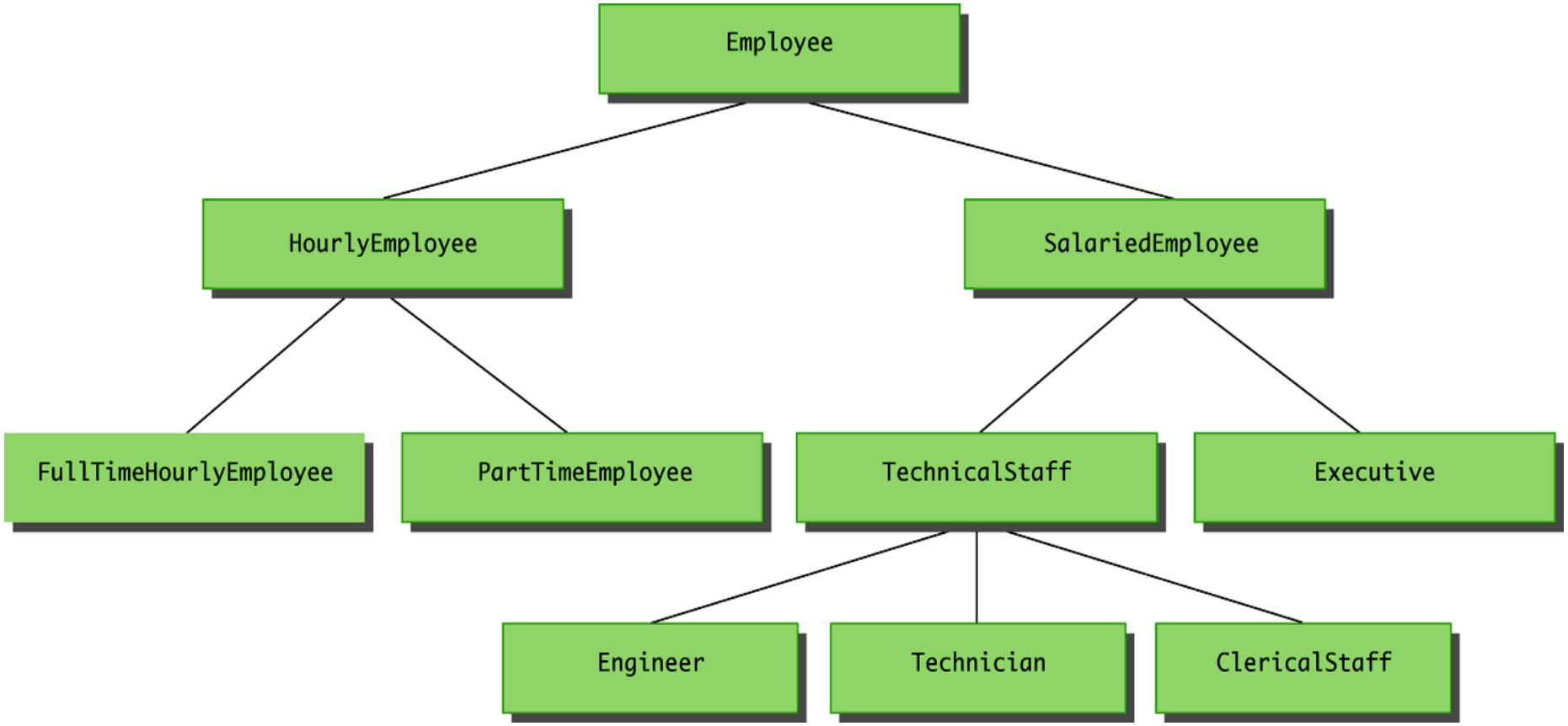
- Μπορούμε να ορίσουμε παράγωγες κλάσεις των παράγωγων κλάσεων
  - Με αυτό τον τρόπο ορίζεται μια ιεραρχία κλάσεων

Ιεραρχία κλάσεων (Class Hierarchy)

- Παράδειγμα: Έχουμε ένα πρόγραμμα που διαχειρίζεται τους Εργαζόμενους μιας εταιρίας
  - Όλοι οι εργαζόμενοι έχουν κοινά χαρακτηριστικά, το όνομα και το ΑΦΜ τους
- Οι εργαζόμενοι χωρίζονται σε Ωρομίσθιους και Έμμισθους
  - Διαφορετικά χαρακτηριστικά θα κρατάμε όσον αφορά το μισθό για τον καθένα
- Οι Ωρομίσθιοι χωρίζονται σε Πλήρους και Μερικής απασχόλησης
- Οι Έμμισθοι χωρίζονται σε Τεχνικό Προσωπικό και Διευθυντικό προσωπικό
- Κ.ο.κ….

A Class Hierarchy

Display 7.1 **A Class Hierarchy**

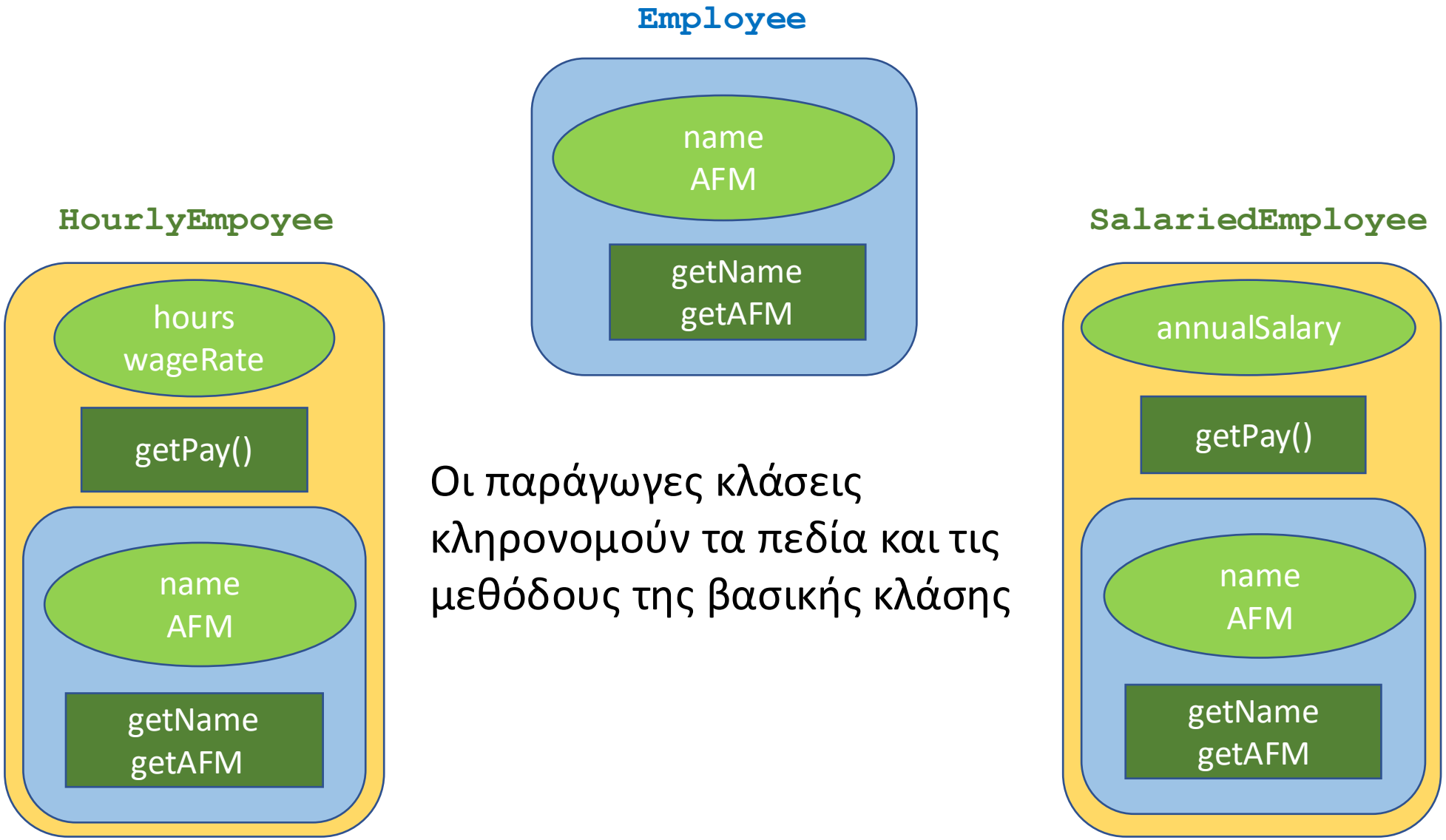


Ιεραρχία κλάσεων

- Η ιεραρχία από κλάσεις ορίζει κάτι σαν γενεαλογικό δέντρο κλάσεων από πιο γενικές προς πιο ειδικές κλάσεις

- Στη Java όλες οι κλάσεις ανήκουν στην ίδια ιεραρχία
  - Στην κορυφή της ιεραρχίας είναι η κλάση Object

Παράδειγμα



Οι παράγωγες κλάσεις κληρονομούν τα πεδία και τις μεθόδους της βασικής κλάσης

Ορολογία

- Η βασική κλάση συχνά λέγεται και **υπέρ-κλάση (superclass)** και η παραγόμενη κλάση **υπό-κλάση (subclass)**
- Επίσης η βασική κλάση λέμε ότι είναι ο **γονέας** της παραγόμενης κλάσης, και η παράγωγη κλάση το **παιδί** της βασικής
  - Αν έχουμε παραπάνω από ένα επίπεδο κληρονομικότητας στην ιεραρχία, τότε έχουμε πρόγονο και απόγονο κλάση

Συντακτικό

- Ας πούμε ότι έχουμε την βασική κλάση **Employee** και τις παραγόμενες κλάσεις **HourlyEmployee** και **SalariedEmployee**
- Για να ορίσουμε τις παραγόμενες κλάσεις χρησιμοποιούμε το εξής συντακτικό στη δήλωση της κλάσης:

```
public class HourlyEmployee extends Employee
public class SalariedEmployee extends Employee
```

Η βασική κλάση

```
public class Employee
{
    private String name;
    private int AFM;

    public Employee( ) { … }

    public Employee(String theName, int theAFM) { … }

    public Employee(Employee originalObject) { … }

    public String getName( ) { … }
    public void setName(String newName) { … }

    public int getAFM( ) { … }
    public void setAFM (int newAFM) { … }

    public String toString() { … }
}
```

Η παράγωγη κλάση HourlyEmployee

```
public class HourlyEmployee extends Employee
{
    private double wageRate;
    private double hours; //for the month

    public HourlyEmployee( ) { … }

    public HourlyEmployee(String theName, int theAFM, double theWageRate,
double theHours) { … }

    public HourlyEmployee(HourlyEmployee originalObject) { … }

    public double getRate( ) { … }
    public void setRate(double newWageRate) { … }

    public double getHours( ) { … }
    public void setHours(double hoursWorked) { … }

    public double getPay( ) {
        return wageRate*hours;
    }

    public String toString( ){ … }
}
```

Νέα πεδία για την HourlyEmployee

Μέθοδος getPay υπολογίζει το μηνιαίο μισθό

```
public class SalariedEmployee extends Employee
{
    private double salary; //annual

    public SalariedEmployee( ) { … }


    public SalariedEmployee(String theName, int theAFM, double theSalary) { … }

    public SalariedEmployee(SalariedEmployee originalObject ) { … }

    public double getSalary( ) { … }
    public void setSalary(double newSalary) { … }

    public double getPay( )
    {
        return salary/12;
    }

    public String toString( ) { … }
}
```

Η παράγωγη κλάση SalariedEmployee

Νέα πεδία για την SalariedEmployee

Μέθοδος getPay υπολογίζει το μηνιαίο μισθό. Διαφορετική από την προηγούμενη

```
public class Example1
{
    public static void main(String[] args)
    {
        HourlyEmployee alice = new HourlyEmployee("Alice", 100, 50.5, 40);

        SalariedEmployee bob = new SalariedEmployee("Bob", 200, 100000);

        System.out.println("Alice: "
                + alice.getName() + " "
                + alice.getAFM() + " "
                + alice.getPay());

        System.out.println("Bob: "
                + bob.getName() + " "
                + bob.getAFM() + " "
                + bob.getPay());
    }
}
```

Μέθοδοι της Employee

Μέθοδοι των παράγωγων κλάσεων

Constructor

```
public class Employee{
    private String name;
    private int AFM;

    public Employee() {
       name = "no name";
       AFM = 0;
    }

    public Employee(String theName, int theAFM){
        if (theName == null || theAFM <= 0){
             System.out.println("Fatal Error creating employee.");
             System.exit(0);
        }
        name = theName;
        AFM = theAFM;
    }
}
```

```
public class HourlyEmployee extends Employee
{
    private double wageRate;
    private double hours; //for the month

    public HourlyEmployee(String theName, int theAFM, double theWageRate, double
theHours)
    {
      super(theName, theAFM);
      if ((theWageRate >= 0) && (theHours >= 0)){
          wageRate = theWageRate;
          hours = theHours;
      }
      else {
         System.out.println("Fatal Error: creating
                             an illegal hourly employee.");
         System.exit(0);
      }
    }
}
```

Με τη λέξη κλειδί **super** αναφερόμαστε στην βασική κλάση

Εδώ καλούμε τον **constructor** της Employee με ορίσματα το όνομα και το ΑΦΜ

Ο constructor **super** μπορεί να κληθεί **μόνο στην αρχή** της μεθόδου

```
public class SalariedEmployee extends Employee
{
    private double salary; //annual

    public SalariedEmployee(String theName, int theAFM, double theSalary){
        super(theName, theAFM);
        if (theSalary >= 0)
            salary = theSalary;
        else {
            System.out.println("Fatal Error: Negative salary.");
            System.exit(0);
        }
    }
}
```

```
public class SalariedEmployee extends Employee
{
    private double salary; //annual

    public SalariedEmployee()
    {
        super();
        salary = 0;
    }
}
```

Καλεί τον **default constructor** της **Employee**

Η εντολή δεν είναι απαραίτητη σε αυτή την περίπτωση. Αν δεν έχουμε κάποια κλήση προς τον constructor της γονικής κλάσης, τότε καλείται εξ ορισμού ο default constructor της Employee

```
public class SalariedEmployee extends Employee
{
    private double salary; //annual

    public SalariedEmployee(String theName,int theAFM)
    {
        salary = 0;
    }
}
```

Πως θα αρχικοποιηθεί το αντικείμενο στην περίπτωση που κληθεί αυτός ο constructor?

Εφόσον δεν καλούμε εμείς κάποιο constructor της γονικής κλάσης θα κληθεί ο default constructor ο οποίος θα αρχικοποιήσει το όνομα στο “no name” και το ΑΦΜ στο μηδέν

```
public class SalariedEmployee extends Employee
{
    private double salary; //annual

    public SalariedEmployee(String theName,int theAFM)
    {
        super(theName, theAFM);
        salary = 0;
    }
}
```

Αν θέλουμε να αρχικοποιήσουμε το όνομα και το ΑΦΜ θα πρέπει να καλέσουμε τον αντίστοιχο constructor της γονικής κλάσης

Constructor this

- Όπως καλείται ο constructor **super** της γονικής κλάσης μπορούμε να καλέσουμε και τον constructor **this** της ίδιας κλάσης

```
public class SalariedEmployee extends Employee
{
    private double salary; //annual

    public SalariedEmployee(String theName, int theAFM, double theSalary)
    {
         super(theName, theAFM);
         if (theSalary >= 0)
             salary = theSalary;
         else{
            System.out.println("Fatal Error: Negative salary.");
             System.exit(0);
         }
    }

    public SalariedEmployee(){
        this("no name", 0, 0);
    }
}
```

Καλεί ένα άλλο constructor της ίδιας κλάσης

Γιατί να μην κάνουμε κάτι πιο απλό? Κατευθείαν ανάθεση των πεδίων

```
public class SalariedEmployee extends Employee
{
    private double salary; //annual

    public SalariedEmployee(String theName,
                            int theAFM, double theSalary)
    {
        name = theName;
        AFM = theAFM;
        salary = theSalary;
    }
}
```

ΛΑΘΟΣ!

Οι παραγόμενες κλάσεις **δεν** έχουν πρόσβαση στα **private** πεδία και τις **private** μεθόδους της βασικής κλάσης

Κληρονομικότητα και ενθυλάκωση

- Οι παραγόμενες κλάσεις κληρονομούν την πληροφορία που έχει και η γονική κλάση
  - Ένα αντικείμενο **SalariedEmployee** έχει πληροφορία για το όνομα και το ΑΦΜ του υπαλλήλου
- **Δεν έχουν** όμως **πρόσβαση** να διαβάσουν και να αλλάξουν ότι είναι **private** μέσα στην γονική κλάση
  - Στην περίπτωση του **SalariedEmployee**, δεν μπορούμε να αλλάξουμε ή να διαβάσουμε το όνομα. Θα πρέπει να χρησιμοποιήσουμε τις **public μεθόδους setName, getName**
  - Για τον constructor πρέπει να καλέσουμε την super
- Με αυτό τον τρόπο προστατεύουμε τα δεδομένα της γονικής κλάσης από κώδικα εκτός της κλάσης
- Ο περιορισμός ισχύει και για μεθόδους που είναι private στην γονική κλάση

```
public class Employee
{
      private void doSomething(){
            System.out.println("doSomething");
      }
}
```

```
public class SalariedEmployee extends Employee
{
      public void doSomethingMore(){
            doSomething();
            System.out.println("and more");
      }
}
```

ΛΑΘΟΣ!

Protected μέλη

- Οι παράγωγες κλάσεις έχουν **πρόσβαση** σε όλα τα **public** πεδία και μεθόδους της γενικής κλάσης
- **ΔΕΝ** έχουν πρόσβαση στα private πεδία και μεθόδους
  - Μόνο μέσω public μεθόδων **set*** και **get***
- **Protected**: αν κάποια **πεδία** και **μέθοδοι** είναι **protected** μπορούν να τα δουν όλοι οι απόγονοι της κλάσης
- **Package access**: αν δεν προσδιορίσετε **public**, **private**, ή **protected** access τότε η **default** συμπεριφορά είναι η μεταβλητή να είναι προσβάσιμη από τις άλλες κλάσεις μέσα στο ίδιο πακέτο

Employee

```
public class Employee
{
    private String name = "default";
    private Date hireDate = new Date(11,4,2013);

    public Employee() {
       System.out.println("empty contructor");
    }

    public Employee(String theName, Date theDate){
       name = theName;
       hireDate = new Date(theDate);
    }
}
```

```
public class HourlyEmployee extends Employee
{
    private double wageRate;
    private double hours; //for the month


    public HourlyEmployee(String theName, Date theDate,
                          double theWageRate, double theHours)
    {
        name = theName;
        hireDate = new Date(theDate);
        wageRate = theWageRate;
        hours = theHours;
    }
}
```

Χτυπάει λάθος η πρόσβαση σε private πεδία

Employee

```
public class Employee
{
    protected String name = "default";
    protected Date hireDate = new Date(11,4,2013);

    public Employee() {
      System.out.println("empty contructor");
    }

    public Employee(String theName, Date theDate) {
      name = theName;
      hireDate = new Date(theDate);
    }
}
```

```
public class HourlyEmployee extends Employee
{
    private double wageRate;
    private double hours; //for the month


    public HourlyEmployee(String theName, Date theDate,
                          double theWageRate, double theHours)
    {
        name = theName;
        Date = new (theDate);
        wageRate = theWageRate;
        hours = theHours;
    }
}
```

ΟΚ η πρόσβαση σε protected πεδία

Υπέρβαση μεθόδων (method overriding)

- Μία μέθοδος που ορίζεται στην βασική κλάση μπορούμε να την **ξανα-ορίσουμε** στην παράγωγη κλάση με διαφορετικό τρόπο
  - Παράδειγμα: η μέθοδος `toString()`. Την ξανα-ορίζουμε για κάθε παραγόμενη κλάση ώστε να παράγει αυτό που θέλουμε
  - Αυτό λέγετε υπέρβαση της μεθόδου (method overriding)
- Η **υπέρβαση** των μεθόδων είναι διαφορετική από την **υπερφόρτωση**.
  - Στην υπερφόρτωση **αλλάζουμε την υπογραφή** της μεθόδου.
  - Εδώ έχουμε την ίδια υπογραφή, απλά **αλλάζει ο ορισμός** στην παραγόμενη κλάση.

```
public class Employee
{
    private String name;
    private int AFM;

    public Employee( ) { … }

    public Employee(String theName, int theAFM) { … }

    public Employee(Employee originalObject) { … }

    public String getName( ) { … }
    public void setName(String newName) { … }

    public Date getAFM( ) { … }
    public void setAFM(int AFM) { … }

    public String toString()
    {
        return (name + " " + AFM);
    }

}
```

```
public class HourlyEmployee extends Employee
{
    private double wageRate;
    private double hours; //for the month

    public HourlyEmployee( ) { … }

    public HourlyEmployee(String theName, int theAFM, double theWageRate, double
theHours) { … }

    public HourlyEmployee(HourlyEmployee originalObject) { … }

    public double getRate( ) { … }
    public void setRate(double newWageRate) { … }

    public double getHours( ) { … }
    public void setHours(double hoursWorked) { … }

    public double getPay( ) {
        return wageRate*hours;
    }

    public String toString( ){
       return (getName( ) + " " + getAFM( )
            + "\n$" + wageRate + " per hour for " + hours + " hours");
    }
}
```

```
public class SalariedEmployee extends Employee
{
    private double salary; //annual

    public SalariedEmployee( ) { … }
    public SalariedEmployee(String theName, int theAFM, double theSalary) { … }
    public SalariedEmployee(SalariedEmployee originalObject ) { … }

    public double getSalary( ) { … }
    public void setSalary(double newSalary) { … }

    public double getPay( )
    {
        return salary/12;
    }

    public String toString( ) { … }
    {
        return (getName( ) + " " + getAFM( )
                                 + "\n$" + salary + " per year");
    }
}
```

```
public class SalariedEmployee extends Employee
{
    private double salary; //annual

    public SalariedEmployee( ) { … }
    public SalariedEmployee(String theName, int theAFM, double theSalary) { … }
    public SalariedEmployee(SalariedEmployee originalObject ) { … }

    public double getSalary( ) { … }
    public void setSalary(double newSalary) { … }

    public double getPay( )
    {
        return salary/12;
    }

    public String toString( ) { … }
    {
        return (super.toString( ) + "\n$" + salary + " per year");
    }
}
```

Έτσι καλούμε την **toString** της βασικής κλάσης
Πιο καλή υλοποίηση, μπορεί να έχει φωλιασμένες κλήσεις από προγονικές κλάσεις

super

- Το keyword **super** χρησιμοποιείται σαν αντικείμενο κλήσης για να καλέσουμε μια μέθοδο της γονικής κλάσης την οποία έχουμε κάνει override
  - Π.χ., **super.toString()** για να καλέσουμε την **toString** της **Employee**
- Αν θέλουμε να το ξεχωρίσουμε από την κλήση της toString της SalariedEmployee, μπορούμε να χρησιμοποιήσουμε το **this**
  - Μέσα στην **SalariedEmployee**:
    - **super.toString()** καλεί την **toString** της **Employee**
    - **this.toString()** καλεί την **toString** της **SalariedEmployee**
- Προσοχή: Δεν μπορούμε να έχουμε αλυσιδωτές κλήσεις του super
  - super.super.toString() είναι λάθος!

Παράδειγμα

```
public class InheritanceDemo
{
    public static void main(String[] args)
    {
        SalariedEmployee sam = new SalariedEmployee("Sam", 100, 100000);
        HourlyEmployee han = new HourlyEmployee("Han", 200, 50.5, 40);
        Employee eve = new Employee("Eve",300);

        System.out.println(eve);

        System.out.println(sam);

        System.out.println(han);
        }
}
```

Καλεί τη μέθοδο της **Employee**

Καλεί τη μέθοδο της **SalariedEmployee**

Καλεί τη μέθοδο της **HourlyEmployee**

Πολλαπλοί τύποι

- Ένα αντικείμενο της παράγωγης κλάσης έχει και τον τύπο της βασικής κλάσης
  - Ένας **HourlyEmployee** είναι **και** **Employee**
  - Υπάρχει μία **is-a** σχέση μεταξύ των κλάσεων

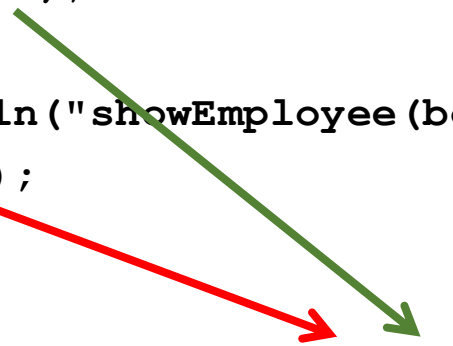
- Αυτό μπορούμε να το εκμεταλλευτούμε χρησιμοποιώντας την βασική κλάση όταν θέλουμε να χρησιμοποιήσουμε κάποια από τις παράγωγες

```
public class IsADemo
{
    public static void main(String[] args){
        HourlyEmployee alice = new HourlyEmployee("Alice", 100, 50.5, 40);
        SalariedEmployee bob = new SalariedEmployee("Bob", 200, 100000);

        System.out.println("showEmployee(alice):");
        showEmployee(alice);

        System.out.println("showEmployee(bob):");
        showEmployee(bob);
    }

    public static void showEmployee(Employee employeeObject)
    {
            System.out.println(employeeObject.getName( ));
            System.out.println(employeeObject.getAFM( ));
    }
}
```



Μπορούμε να καλέσουμε τη μέθοδο και με **HourlyEmployee** και με **SalariedEmployee** γιατί και οι δύο είναι **και** Employee.

```
public class Employee
{
        private String name;
        private int AFM;

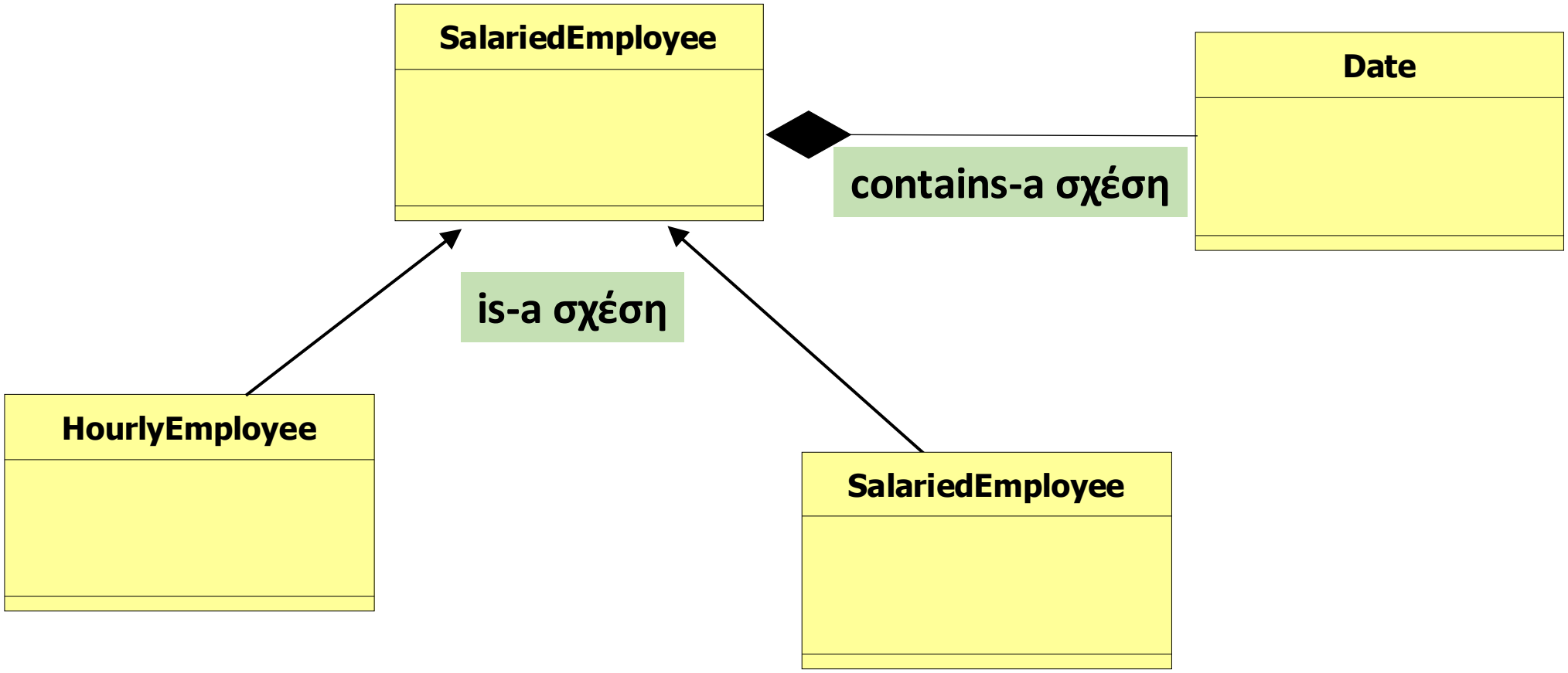
        public Employee(Employee other){
                this.name = other.name;
                this.AFM = other.AFM;
        }
}
```

```
public class SalariedEmployee extends Employee
{
        public SalariedEmployee(SalariedEmployee other){
            super(other);
                this.salary = other.salary;        }
}
```

Η κλήση του **copy constructor** της **Employee** (μέσω της super(other)) γίνεται με ένα αντικείμενο τύπου **SalariedEmployee**. Αυτό γίνεται γιατί SalariedEmployee **«is a»** Employee και το αντικείμενο other έχει **και τους δύο τύπους**.

UML διάγραμμα

- Αναπαράσταση κληρονομικότητας



Υπέρβαση και αλλαγή επιστρεφόμενου τύπου

- Μια αλλαγή που μπορούμε να κάνουμε στην υπογραφή της κλάσης που υπερβαίνουμε (orerride) είναι να αλλάξουμε τον επιστρεφόμενο τύπο σε αυτόν μιας παράγωγης κλάσης
  - Ουσιαστικά δεν είναι αλλαγή αφού η παράγωγη κλάση έχει και τον τύπο της γονικής κλάσης

```
public class Employee
{
        private String name;
        private Date hireDate;

        public Employee createCopy()
        {
          return new Employee(this);
        }
}
```

```
public class HourlyEmployee extends Employee
{
    private double wageRate;
    private double hours; //for the month

    public HourlyEmployee createCopy()
    {
        return new HourlyEmployee(this);
    }
}
```

Ο επιστρεφόμενος τύπος αλλάζει από Employee σε HourlyEmployee στην υπέρβαση. Ουσιαστικά όμως δεν υπάρχει αλλαγή μιας και κάθε αντικείμενο HourlyEmployee είναι και Employee

```
public class SalariedEmployee extends Employee
{
    private double salary; //annual

    public SalariedEmployee createCopy()
    {
        return new SalariedEmployee(this);
    }

}
```

Ο επιστρεφόμενος τύπος αλλάζει από Employee σε SalariedEmployee στην υπέρβαση. Ουσιαστικά όμως δεν υπάρχει αλλαγή μιας και κάθε αντικείμενο SalariedEmployee είναι και Employee